Memory-Centric Agentic Inference Final Report

2026-05-12

Contents

Memory-Centric Agentic Inference Final Report

Abstract

The research package supports a memory-centric architecture thesis for agentic large language model (LLM) inference as a conditional mechanism stack, not as an unconditional production recommendation. Agentic inference means LLM execution that can call tools, preserve workspace state, branch into alternative paths, verify intermediate results, merge work from multiple agents, and resume or replay prior state. In that setting, the relevant infrastructure problem is not only arithmetic throughput. It is also the movement, placement, reuse, compression, security, and lifetime management of model weights, key-value (KV) cache, prompt prefixes, retrieved context, semantic-cache entries, tool outputs, verifier state, branch state, trajectory logs, and durable workspace state.

The validated result is a three-option architecture frame. Option A is conventional request/model/KV-centric serving and remains the default for controls, single-turn chat, batch/offline inference, zero-reuse regimes, and cases where coordination overhead would erase retained-state value. Option B is a memory-object-aware runtime for object-local reuse such as retrieved context, prefix/cache state, semantic-cache entries, and tool outputs when provenance, freshness, isolation, invalidation, and validation gates pass. Option C is a trajectory/DAG-aware memory fabric for branch state, verifier state, trajectory logs, durable workspace state, lineage, and multi-agent merge state when retained value survives queueing, validation, security, compression, recovery, and durable-tail costs. A directed acyclic graph (DAG) is the dependency graph of branch, verifier, replay, merge, discard, and durable-artifact relationships in an agent run.

The final structured audit reports 89 validated milestones, 2 superseded milestones, and 1 in-progress sentinel, with findings counted as 1 critical, 14 moderate, and 2 minor. The same audit records `promise_check=red`, not because the architecture thesis was rejected, but because residual governance and packaging defects remain: a malformed ledger event, stale periodic-report statements, an incomplete handoff index, incomplete registered

package archives, and an in-progress run-start sentinel. The available record is not marked as wall-cap truncated. The report therefore treats the technical package as broadly validated at the mechanism level while preserving the boundary that no Option B or Option C claim is production-ready without real joined `production_target` telemetry that passes the full evidence chain.

Introduction

The central question is whether future AI infrastructure for agentic inference should be organized around memory state rather than arithmetic throughput alone. Existing LLM serving already faces pressure from memory capacity, memory bandwidth, data movement, interconnects, and power. Agentic systems add another layer: they can create persistent context, reuse earlier tool results, keep verifier evidence, branch and merge work, replay durable workspace artifacts, and coordinate multiple agents over long horizons. These behaviors make memory state a first-class systems object rather than a passive side effect of token generation.

This report is a synthesis of the completed research package. It is not a generic survey. The package built a workload and memory-object taxonomy, symbolic lifetime and heterogeneous-memory cost models, a deterministic simulator, scheduling-abstraction comparisons, a trace schema, queueing and compression boundaries, a toy runtime prototype, a sourced calibration map, security and provenance gates, measurement designs, production-evidence contracts, and final readiness artifacts. The report states what is sourced from public references, what is derived analytically, what is simulated, what is a synthetic fixture or proxy, and what remains unmeasured or speculative.

A memory object is a first-class unit of retained or movable state. Examples include model weights, KV cache, prefix cache, retrieved context, tool output, intermediate scratch, branch state, verifier state, trajectory log, durable workspace state, and semantic-cache entries. Each object has a lifetime: it is born, accessed, updated, placed in some tier, possibly migrated or compressed, and eventually evicted, merged, made durable, or reused. The central modeling move in the package was to ask which of those events determine future value. A discarded KV cache may only force token recomputation; a discarded verifier trace may erase a counterexample; a stale semantic-cache entry may create an incorrect answer; a lost durable workspace pointer may break replay or auditability.

The package separates three evidence levels. Sourced evidence comes from external hardware, interconnect, storage, benchmark, and systems references. Derived evidence comes from formulas or symbolic models, such as lifetime equations and queueing reversal inequalities. Simulated evidence comes from deterministic synthetic workload, policy, trace, queueing, compression, and sensitivity harnesses. Synthetic fixtures and host-local proxies test schemas, fail-closed behavior, and threshold plumbing, but they do not provide production calibration. Production endorsement is reserved for real joined production telemetry that passes schema, join, noise-floor, security, provenance, retention, verifier, causal-control, threshold, calibration, and claim-expiry gates.

The resulting answer is conditional. Future infrastructure should be memory-centric when retained state value exceeds the costs of exposing and managing that state. The same package also defines when the thesis should collapse. If useful state is mostly weights, active KV cache, prefix cache, and transient scratch, Option A is the correct low-overhead boundary. If object-local reuse matters but branch dependencies do not, Option B is the target. If branch survival, verifier reuse, durable replay, trajectory lineage, or multi-agent merge state determines future value, Option C becomes the candidate. In all cases, the memory-centric option must lose when metadata queues, validation overhead, security risk, stale provenance, compression loss, or durable-tail latency exceeds retained value.

Architecture Options and Memory Objects

The architecture proposal has three options. They are not ranked from old to new; they are compatibility boundaries for different workload regimes.

Option	Runtime boundary	Primary memory objects	Appropriate use
Option A: conventional request/model/KV-centric serving	Request, model, cache page, and active context	Model weights, KV cache, prefix cache, intermediate scratch	Single-turn chat, batch/offline inference, cheap recomputation, zero-reuse workloads, and controls.
Option B: memory-object-aware runtime	Addressable object registry with object-local placement and retention	Retrieved context, prefix/cache objects, semantic-cache entries, tool outputs, provenance pointers	Retrieval-augmented generation and tool workflows where object identity, reuse, provenance, freshness, and invalidation determine value.
Option C: trajectory/DAG-aware memory fabric	Agent trajectory graph with branch, verifier, replay, merge, and durable dependencies	Branch state, verifier state, trajectory logs, durable workspace state, replayable tool outputs, lineage state	Code-agent loops, verification-heavy agents, and multi-agent branch/merge runs where future value depends on cross-object dependencies.

Option A is deliberately preserved. The simulator and scheduling evaluations included control workloads, and those controls favored coarse serving boundaries. This matters because the research question is not answered by assuming every workload benefits from richer state management. When memory-centric metadata exposes no causal value, it becomes overhead.

Option B adds a memory-object registry. A registry makes objects addressable by class, size, lifetime, reuse probability, recomputation cost, correctness sensitivity, provenance pointer, invalidation signal, and placement tier. That visibility is sufficient for workloads where the main question is whether a specific object should be retained, compressed, offloaded, recomputed, or invalidated. RAG-like workloads fall in this category when retrieved context and semantic-cache entries can be reused safely. Tool outputs can also fit this boundary when they are object-local and do not depend on branch or verifier graph structure.

Option C adds trajectory structure. A trajectory is the graph of an agent run: actions, tool calls, branches, verifier loops, counterexamples, merges, discarded paths, replay points, and durable workspace writes. Option C is useful only when object-local information is insufficient. For example, a verifier artifact may be valuable because it blocks a bad branch; a durable workspace file may be valuable because several later tasks depend on it; a trajectory log may be valuable because it preserves auditability or lets a future agent resume the run. These are graph-level facts, not just object-local cache facts.

The memory hierarchy considered by the package spans hot accelerator memory, GPU-to-GPU fabric, host CPU DRAM, PCIe links, CXL or pooled memory, local NVMe, remote storage or object stores, semantic and prefix cache services, and durable workspace state. The package does not claim calibrated performance across those tiers. Instead, it defines which variables must be measured: capacity, bandwidth, latency, transfer cost, energy or dollar cost per byte moved or retained, contention behavior, durability tail latency, validation overhead, and safe reuse probability.

The design principle that ties the options together is:

```
Expose the coarsest memory-state boundary that preserves the causal variables needed for retained value.
```

For controls, that boundary is conventional serving. For object-local reuse, it is the memory object. For branch-heavy, verifier-heavy, durable, or multi-agent runs, it may be the trajectory/DAG. The same principle also defines the falsification path: if a finer boundary does not preserve additional value after overhead and safety gates, it should collapse back to the coarser option.

Validated Mechanism Stack

The mechanism stack starts with a workload taxonomy, then adds lifetime equations, tier-cost accounting, synthetic policy comparison, scheduling-boundary evaluation, trace replay, queueing reversal tests, compression safety rules, runtime ablation, calibration mapping, and security-adjusted synthesis. Each layer keeps the same evidence boundary: the package validates mechanisms and measurement contracts, not production savings.

The taxonomy defines nine workload classes and eleven memory-object classes. The workload classes include single-turn chat, multi-turn chat, batch summarization, retrieval-augmented generation (RAG), code-agent loops, tool-using research agents, verification-heavy agents, multi-agent branch/merge runs, and offline inference. The object classes are weights, KV cache, prefix cache, retrieved context, tool output, intermediate scratch, branch state, verifier state, trajectory log, durable workspace, and semantic-cache entry. This taxonomy is the first substantive result because it separates ordinary serving state from state whose value depends on provenance, branch survival, verification, replay, durability, or auditability.

The lifetime model represents a memory object as:

```
O_i = {class_i, S_i(t), b_i, e_i, R_i(t), P_i(t), C_evict_i}
```

where `S_i(t)` is size over time, `b_i` and `e_i` are birth and end events, `R_i(t)` is reuse behavior, `P_i(t)` is placement, and `C_evict_i` is the consequence of eviction. Expected live bytes are:

```
E[L_i(t)] = E[S_i(t) * 1(b_i <= t < e_i)]
```

The retained-value expression is:

```
V_i(t) =
  Pr(reuse_i before eviction) * C_recompute_i
  + Pr(correctness_sensitive_i) * C_loss_i
  - C_residency_i
```

This model makes the object boundary concrete. KV cache grows with token count and context cap. Prefixes depend on repeated exact reuse. Branch state depends on fanout, survival probability, verifier delay, and merge probability. Durable workspace state grows with artifact rate and retention horizon, and is not automatically bounded by the model context window.

The heterogeneous cost model adds tier placement and movement terms across accelerator memory, host DRAM, CXL or pooled memory, NVMe, remote storage, durable workspace stores, and semantic/prefix-cache services. Its useful-retention inequality is:

```
P_reuse_i * C_recompute_i + P_correct_i * C_loss_i
  >
C_residency(i,k) + C_transfer(i,k) + C_bandwidth_delay(i,k)
```

The initial tier values, energy proxies, and dollar proxies are symbolic or synthetic placeholders. The result is therefore structural: it identifies the variables a policy must compare, not calibrated cost savings.

The deterministic simulator then tested four policies across six synthetic regimes. The controls selected `hbm_first_baseline` and weakened the memory-centric thesis. RAG selected `reuse_aware_tiering`, making the object-local case plausible but still sensitive to provenance and invalidation. Code-agent, verification-heavy, and branch/merge regimes selected `branch_verifier_durable_aware`, strengthening the thesis for non-KV state. Object attribution showed that the agentic wins were driven by durable workspace, tool output, branch state, verifier state, and trajectory logs, not by KV cache alone.

The scheduling-abstraction evaluation interpreted each candidate unit as an information boundary. It compared request, job, kernel, model, cache page, context segment, memory object, and agent trajectory DAG. The synthetic benefit expression was:

```
Benefit(unit, workload) =
  observed retained value
  + movement avoidance
  + reuse capture
  + branch capture
```

```
+ durability capture
+ correctness capture
- coordination overhead
```

The winners matched the architecture frame: model or cache-page boundaries for controls, memory-object scheduling for RAG, and trajectory/DAG scheduling for code-agent, verification-heavy, and multi-agent branch/merge workloads. The scheduling result is not that richer scheduling always wins. It is that the correct scheduling boundary is the coarsest one that still exposes the state variables that determine retained value.

The trace schema made those variables observable. The synthetic trace v2 includes object creation, access, update, placement, migration, eviction, recomputation, branch fork/merge/discard, verifier start/result, tool calls, workspace writes, semantic-cache lookup/insert, provenance, invalidation, correctness sensitivity, and run boundaries. It generated 503 synthetic events across six workloads. The workload summaries preserved the architecture split: controls mapped to Option A with zero non-KV share; RAG mapped to Option B with object reuse but no trajectory width; code-agent, verification-heavy, and branch/merge workloads mapped to Option C with non-KV shares around 0.46 to 0.55, DAG width, and verifier results.

The queueing model added the principal reversal condition. It uses a simple M/M/1 proxy to represent metadata, policy, migration, DAG coordination, verifier synchronization, durable consistency, and preemption/checkpoint paths:

```
rho_s = lambda_s / mu_s
Wq_s = rho_s / (mu_s - lambda_s)
Q_s = ops_s * Wq_s
```

The threshold results are the main point. Option B beats Option A only when object reuse benefit exceeds registry, object-policy, and migration queues. Option C beats Option B only when branch, verifier, and durable benefit exceed DAG, verifier-sync, durable-consistency, and preemption queues. In the synthetic winner table, high object overhead collapses RAG and agentic workloads toward Option A, while high DAG overhead collapses agentic workloads from Option C to Option B.

Compression and offload were narrowed into a representation-validity mechanism. The valid strategies include keeping state hot, lossless compression, lossy summarization, summary plus pointer, full offload, and recompute on demand. Unsafe lossy strategies are rejected before ranking when they would destroy replay, provenance, correctness, recovery, verifier, branch, trajectory, tool-output, semantic-cache, or durable-workspace semantics. A later repair removed unsupported claims that compression helped avoid queueing reversal thresholds under the current synthetic coefficients. The validated claim is narrower: compression/offload supports capacity, movement, local storage, provenance preservation, and safety; it is not presently evidence that Option B or Option C survives hot-path queue saturation.

The runtime prototype made the option boundary executable. It consumes the synthetic trace and prior outputs, builds an object registry, and emits placement, retention, compression, and eviction decisions. The registry tracks object class, workload, size, tier, lifetime, reuse count and distance, correctness sensitivity, provenance, source version, invalidation, trajectory node, branch ID, verifier ID, durability horizon, merge state, and policy decisions. The prototype reproduced the expected A/B/C labels for all six workloads. Its ablations are the important mechanism check: hiding provenance and reuse collapses RAG from Option B to Option A; hiding branch, verifier, and durable fields collapses agentic workloads from Option C toward Option B; hiding all memory-causal fields collapses non-controls to Option A.

The calibration map then separated public evidence from synthetic carryover. Public sources support the reality of accelerator memory pressure, GPU and host memory tiers, PCIe/NVMe/CXL capability framing, KV-cache management, prefix caching, semantic-cache mechanisms, and benchmark context [1]-[14]. They do not provide production measurements for trajectory reuse distributions, provenance-validation overhead, semantic-cache safe-hit rates, durable replay tails for agentic state, or CXL/pooled-memory contention under agentic workload pressure. Those are deferred constants, not inferred facts.

The security/provenance layer made safe reuse a precondition for counting retained value. Its decision rule is:

```
AuthorizedReuse(object, actor, source_version, lineage, invalidation_state) = true

SecurityAdjustedValue =
```

```
  RetainedValue
  - CoordinationOverhead
  - ValidationOverhead
  - ExpectedSecurityLoss
```

Option A mainly needs cache isolation and cleanup. Option B adds provenance, source freshness, invalidation, tenant/cache-salt partitioning, and cache-poisoning checks. Option C adds lineage, replay authorization, verifier evidence integrity, retention policy, deletion/audit conflicts, and durable replay authorization. The synthetic security model showed that validation and expected security loss can reverse apparent retained value for RAG and agentic workloads, so security is part of architecture selection rather than a separate deployment footnote.

The integrated synthesis rule is:

```
ArchitectureChoice(w) =
  argmax over A/B/C of
    visible retained value
  + movement/correctness benefit
  - coordination cost
  - compression/recovery cost
  - validation overhead
  - expected security loss

subject to authorized reuse for every retained object.
```

Under the current synthetic and derived package, the final workload decisions remain Option A for controls, Option B for RAG, and Option C for code-agent, verification-heavy, and multi-agent branch/merge workloads. Those decisions are mechanism-level decisions. They remain sensitive to the deferred measurements below.

Measurement Designs and Deferred Constants

The measurement layer converts the speculative parts of the architecture thesis into named deferred constants, required fields, experiment rows, thresholds, and claim-update rules. A deferred constant is a quantity that the package cannot fill from public sources or synthetic traces without overclaiming. It can support, downgrade, or falsify an architecture option once measured.

`DC-001` is per-tier energy and dollar cost per byte moved or retained. The energy/economics harness defines:

```
NetEnergyValue =
  E_recompute_avoided
  + E_movement_avoided
  - E_residency
  - E_transfer
  - E_validation
  - E_coordination
```

The current result is a sensitivity harness, not measured energy savings. It defines what production telemetry must collect before energy or cost claims can be promoted: per-tier joules per byte moved and retained, dollar cost per retained and moved byte, and safe reuse rate before energy credit is counted.

`DC-002` is CXL or pooled-memory latency under contention. The harness emits threshold rows showing when warm-tier placement should be downgraded because p50, p95, or p99 latency tails exceed retained-value margins. The synthetic sweep keeps the CXL claim bounded: CXL or pooled memory is a candidate tier only when measured contention tails stay below the benefit margin for the relevant object group.

`DC-003` is durable-state replay risk. It covers durable workspace objects, checkpoints, summary pointers, tool outputs, verifier artifacts, and trajectory logs that may be read later for replay or merge. The measurement design defines:

```
V_durable_replay =
  P_replay * C_recompute_avoided
  - L_store_read(p)
```

```
  - L_consistency(p)
  - C_pointer_validation
  - C_reconstruction
  - E[C_replay_failure]
```

The percentile `p` must include p50, p95, and p99 because an agentic replay path can block on the slowest object in a dependency chain. Option C is not supported by median durable-read latency; it requires tail-safe replay value after dependency fan-in, pointer validity, consistency waits, retention validity, and recovery cost.

`DC-004` is semantic-cache correctness and invalidation cost. A semantic-cache hit only counts when it is a valid hit: fresh, provenance-valid, tenant-safe, cache-salt-safe, not poisoned, recoverable, and correct enough for the workload. The measurement design defines:

```
V_semantic_safe =
  P_hit * P_valid_given_hit * C_recompute_avoided
  - C_lookup
  - C_validation
  - C_invalidation
  - C_recovery
  - E[C_wrong_reuse]
```

Option B is therefore not justified by raw semantic hit rate. It requires positive safe value after lookup, validation, invalidation, recovery, and wrong-reuse costs. The integrated parent harness includes semantic experiments for raw versus valid hit rate, false-positive rate, stale-hit rate, tenant/cache-salt enforcement, recovery latency, and poisoning or untrusted-provenance rejection.

`DC-005` is production trajectory reuse. It tests whether branch state, verifier state, tool-output replay, trajectory logs, and durable workspace dependencies create enough retained value to justify Option C. The measurement design defines:

```
NetTrajectoryValue =
    V_branch
  + V_verifier
  + V_tool_replay
  + V_trajectory_log
  + V_durable_workspace
  - Q_dag
  - Q_verifier_sync
  - Q_durable_consistency
  - Q_preemption
  - ValidationOverhead_trajectory
  - ExpectedSecurityLoss_trajectory
```

Option C remains justified only when this value exceeds the Option B or Option A fallback. The integrated harness contains `TRJ-001` through `TRJ-007`, covering branch survival, verifier reuse, tool-output replay, durable dependencies, trajectory reuse distance, field-ablation replay, and control negatives. The merge verifier protects the required trajectory IDs, collapse thresholds, required fields, and the rule that unauthorized, stale, tampered, or retention-invalid replay receives zero positive retained value.

`DC-006` is provenance-validation overhead. The artifacted plan covers provenance pointer lookup, source-version freshness, tenant/cache-salt isolation, trajectory lineage validation, replay actor authorization, verifier evidence binding, retention hold-state compliance, and summary-pointer recoverability. The current parent harness records DC-006 as a design gap where reported: the plan exists, but earlier parent integration did not include DC-006 rows at that point. Provenance overhead remains a required measurement before promoting Option B or Option C claims, not a value assumed by the architecture.

Together, the measurement designs define how the package moves from mechanism validation to production evidence. They also define the downgrade path. Option B collapses when semantic safe value, provenance validation, or object metadata overhead cannot pay for reuse. Option C collapses when trajectory reuse, durable

replay, verifier reuse, security validation, or DAG coordination cannot pay for their overheads. Energy and economics claims remain speculative until DC-001/DC-002 telemetry replaces synthetic proxies.

Energy, Economics, Security, and Compression Boundaries

The cross-cutting boundary is that no reused byte receives credit until it is useful, authorized, fresh, recoverable, and cheaper than its management overhead. This rule applies to energy, dollar cost, latency, capacity, correctness, and replay.

For energy and economics, the current package provides falsification machinery rather than savings claims. The DC-001/DC-002 harness produced synthetic scenario, architecture-sensitivity, CXL-threshold, claim-update, and measurement-requirement rows. In the synthetic sensitivity table, Options B and C survive only in regions where retained value remains positive after byte-energy, dollar, residency, transfer, validation, coordination, and CXL contention charges. If per-byte energy or dollar savings are measured as zero or below noise, energy and economics cannot support the memory-centric thesis even if capacity or correctness arguments remain. If CXL or pooled-memory p95/p99 tails exceed retained-value margins, warm-tier placement must be downgraded.

Compression has a different boundary. It is not a generic queue-relief argument. The repaired compression model shows no selected positive object-level queue-help rows under the current coefficients, and 16 queue-harm rows. The valid use of compression/offload in the package is representation-safe state management: keep exact state hot when necessary, losslessly compress exact state when reversible, use summary-plus-pointer only when the pointer preserves provenance and recovery, offload full state when replay tolerates the tier, and recompute only when recomputation does not destroy correctness, auditability, or durable dependency value.

Security is treated as an architecture-selection variable because unsafe reuse can invert the result. The trace-v3 enforcement replay adds tenant scope, cache salt, actor identity, replay authorization, verifier evidence hashes, retention state, pointer validity, validation gates, and validation timing. It produced 268 enforcement decisions and recomputed architecture options from safe reuse credit rather than raw reuse credit. Invalid fixtures go through the same decision path as ordinary replay rows. Missing provenance, stale sources, invalidation signals, tenant or cache-salt mismatch, unauthorized actors, contaminated lineage, tampered verifier evidence, expired retention without hold, invalid pointers, and missing validation timing deny or downgrade reuse.

The security replay preserved RAG as Option B and code-agent and multi-agent branch/merge as Option C under its synthetic assumptions, but it collapsed verification-heavy from Option C to Option A after enforcement. That result is a mechanism warning, not a production claim: verification-heavy workloads are a high-value falsification target because raw trajectory value can disappear once verifier integrity, replay authorization, retention, and validation overhead are counted.

The resulting economic and safety rule is:

```
MemoryCentricArchitecture is strongest when
  RetainedStateValue
  >
  CoordinationCost
  + ValidationOverhead
  + RecoveryCost
  + ExpectedSecurityLoss
  + DurableTailCost
  + EnergyDollarContentionCost
```

This rule explains why Option A remains a first-class outcome. If a workload has no positive safe non-KV retained value, or if the value is erased by queueing, validation, security, compression recovery, durable replay tails, or energy/contention cost, the correct architecture is the conventional serving boundary. Option B and Option C are design targets only when measured retained-state value survives those gates.

Production Evidence Chain

The post-mechanism work turned the architecture package into a gated production-evidence path. The central rule is that synthetic, host-local, fixture, conformance, dry-run, or planned telemetry can validate schemas and

fail-closed behavior, but it cannot create production calibration, production readiness, or claim credit. Each gate adds a condition that future `production_target` evidence must satisfy before it can update DC-001/DC-002 energy and contention claims or promote Option B/C architecture recommendations.

The chain starts with host-local proxy measurement and a production telemetry contract. `M-DC12-1` exercised DC-001 and DC-002 threshold plumbing with local byte-movement and contention measurements. DC-001 is the per-tier energy or dollar cost of bytes moved or retained; DC-002 is the p50/p95/p99 latency of CXL or pooled-memory tiers under contention. The outputs were labeled `host_local_proxy` and `production_calibrated=false`, so they validated measurement flow without calibrating accelerator, CXL, or production energy claims. `M-PRODTELEM-1` then defined the minimum production telemetry schema. A production-calibrated row must have `evidence_label=production_target`, complete required fields, valid join keys, aligned power and byte intervals, deltas above noise, passing security/provenance/retention/verifier gates, and comparable threshold context. Invalid or blocked rows grant zero reuse and energy credit, while valid synthetic production-shaped rows remain candidate-only.

The deployment and trend layer made that contract operational and falsifiable. `M-PRODDEPLOY-1` maps the production telemetry contract into collector categories, join keys, fail-closed preflight checks, and a minimal pilot. A usable production row must join power counters, tier-specific byte counters, CXL or pooled-memory latency, queue depth, tenant concurrency, workload and object labels, reuse and architecture decisions, security/provenance/retention/verifier gates, interval metadata, and noise floors. Planned telemetry remains a collection plan, not measured evidence. `M-TRENDS-1` adds a synthetic future-trend harness that identifies conditions under which Option A, B, or C would be favored or falsified as HBM, CXL/pool latency, durable-state latency, energy per byte, recompute cost, validation overhead, branch fanout, reuse probability, and verifier loops change. Those trend rows preserve `production_ready=false`.

The adapter and intake layer prevents malformed operator telemetry from entering the production path. `M-PORT-1` checks whether backend-shaped telemetry can be canonicalized into the production schema. It validates logical join aliases, required units, clocks, interval alignment, tenant labels, security context, provenance freshness, and fixture evidence boundaries. Passing conformance means a stream shape is interpretable; it does not grant `production_target` status. `M-INTAKE-1` defines the front-door telemetry bundle: manifest schema, payload inventory, row counts, checksums, join windows, provenance, measurement quality, security/privacy declarations, boundary labels, admission outputs, downstream-boundary rows, and traceability. A repaired checksum gate hashes payload files and blocks mismatches. Structural intake admission remains weaker than production calibration.

The trust and promotion layer separates mechanical validity from production trust. `M-TRUSTPOL-1` defines the operator policy needed before fixture signing can be replaced by production KMS, HSM, hardware-attestation root, or operator certificate-authority evidence. The required controls include non-exportable custody, rotation, revocation, collector identity binding, replay protection, tenant/security binding, and auditability. A complete fixture policy can be policy-admissible but still has no trusted production attestation, production calibration, readiness, or claim credit. `M-GATECHAIN-1` composes prior gates into a 14-state promotion path from raw bundle observation through attestation, trust policy, intake, adapter conformance, normalization, production ingestion, security/provenance, noise floor, threshold replay, planner eligibility, final readiness, handoff traceability, and production claim credit. A repaired replay rule requires `state_passed == true`, not mere state presence. All 19 replay paths in that artifact keep `production_claim_credit_allowed=false`.

The measurement-quality layer adds temporal and privacy constraints. `M-TIMEBASE-1` treats malformed timing as `measurement_invalid`, not as a failed threshold result. DC-001/DC-002 replay requires joined source identity, measurement-run identity, collector identity, schema identity, known clock domain, aligned power/byte/latency/queue/security intervals, bounded skew and jitter, bounded observer overhead, fresh workload labels, continuous counters, and bounded drift. `M-REDACT-1` requires telemetry minimization to preserve both privacy and replay-identifiable joins. Under-redacted exports fail as `privacy_leakage`; over-redacted or malformed exports fail as `replay_nonidentifiable`. Stable tenant, object, run, bundle, collector, security-context, and clock-domain pseudonyms are needed so redacted evidence can still join to threshold, readiness, and handoff records.

The scientific-validity and replay layer blocks robust-looking but non-causal claims. `M-CAUSAL-1` requires a valid Option A control arm before threshold results can become readiness evidence. It checks pre-treatment confounders such as workload mix, object size, tenant concurrency, hardware topology, model/runtime version, cache warmness,

security-deny rate, time-window load, and scheduler pressure. Robust statistical effects are blocked if they are causally confounded or unidentified. `M-PRODREPLAY-1` then defines the executable replay surface for real `production_target` telemetry. It distinguishes no real telemetry, rejected evidence, and future claim-support candidacy. A repaired over-credit path now requires both `evidence_label=production_target` and an existing evidence artifact path for every passed gate; future manifests cannot self-assert all gate booleans.

The final production-side and lifecycle layer prevents self-attestation and stale claims. `M-EVIDART-1` requires concrete evidence artifacts for every replay gate, with paths, digests, payload fields, identity bindings, measurement windows, and upstream dependency links. `M-LIVECOLLECT-1` maps each replay gate to production-side source material and blocks production artifact emission when operator, root, attestation, time, counter, or per-gate source evidence is missing. In the current workspace it emits only dry-run fixture artifacts. `M-CLAIMEXP-1` states that even a future successful production replay is not evergreen. Claim support expires by time-to-live, is invalidated by identity-breaking deployment changes, and requires fresh production material when workload, scheduler, memory tier, security, uncertainty, or causal-control conditions drift.

The resulting evidence path is intentionally conservative:

```
Production claim support requires:
  real production_target evidence
  + complete intake custody
  + canonical adapter normalization
  + trusted operator/root/attestation material
  + passing gatechain state
  + valid timebase and redaction joins
  + security/provenance/retention/verifier approval
  + above-noise DC-001/DC-002 threshold replay
  + uncertainty qualification
  + causal Option A control validity
  + linked evidence artifacts
  + live collection source material
  + non-expired claim lifecycle state
```

Every gate hardens admission, replay, or lifecycle semantics. None of the gates by itself makes the memory-centric architecture production-ready.

Production Readiness Status

The current readiness status is mechanism-validated and production-contract-ready, but not production-endorsed. Option A remains the validated conventional baseline and control path. Option B remains a contract-ready pathway for object-local reuse when safe retained object value is proved. Option C remains a contract-ready pathway for trajectory/DAG state when branch, verifier, durable, and replay value is proved. No current artifact upgrades Option B or Option C to a production recommendation.

The strongest current production-readiness facts are negative boundaries:

- Host-local DC-001/DC-002 proxy outputs are labeled `host_local_proxy` and `production_calibrated=false`.
- Synthetic production-shaped telemetry can pass schema gates only as candidate evidence; it does not calibrate production claims.
- Adapter conformance, intake admission, trust-policy admissibility, timebase admissibility, redaction admissibility, causal admissibility, live-collector dry runs, and claim-expiry lifecycle fixtures all grant zero production claim credit.
- All 19 end-to-end evidence-gatechain replay paths have `production_claim_credit_allowed=false`.
- The current production-target replay surface reports zero real production-target manifests, zero claim-support candidates, and zero rows with `production_calibrated=true`, `production_ready=true`, or `claim_credit_allowed=true`.
- The live collector blocks production artifact emission without complete production-side source material and currently permits only dry-run fixture artifacts.
- Any future production support would expire or require revalidation after TTL expiry, identity-breaking deployment changes, or workload, scheduler, memory-tier, security, uncertainty, or causal-control drift.

The production-readiness conclusion is therefore not "the architecture failed." It is that the package has drawn a hard line between mechanism validation and production endorsement. The mechanism stack is sufficient to specify what should be measured, how telemetry should be admitted, and how Option B/C claims would be promoted or rejected. The available evidence is not sufficient to assert measured production energy savings, production CXL or pooled-memory contention benefits, production-safe semantic-cache value, production trajectory reuse value, or production-ready Option B/C deployment guidance.

The required upgrade path is real joined `production_target` telemetry under a trusted deployment root. That telemetry must preserve the joins for workload, object, topology, tenant, security context, timebase, redaction, uncertainty, causal control, threshold replay, planner/readiness update, and handoff traceability. It must also include the source evidence artifacts for each gate rather than only self-reported manifest booleans. Until such rows exist and pass the chain, the correct operational stance is:

Architecture option	Current status	Production promotion condition
Option A	Validated baseline and default control	Remains default when safe non-KV retained value is absent, untrusted, unjoinable, or erased by overhead.
Option B	Mechanism-validated, contract-ready	Requires real production evidence that object-local reuse is safe, fresh, authorized, above noise, causally attributable against Option A, and net-positive after validation/security/queueing costs.
Option C	Mechanism-validated, contract-ready	Requires real production evidence that branch, verifier, trajectory, durable, and replay value survives DAG coordination, durable tails, validation, security, uncertainty, causal, and lifecycle gates.

This readiness boundary is part of the architecture result. A memory-centric design is useful only if the infrastructure can prove that retained state is valuable, safe, current, causally attributable, and cheaper than its management costs. The current package provides the models, harnesses, and gates for that proof. It does not yet contain the real production measurements that would complete it.

Runtime, Compiler, and Control Plane Implications

The runtime implication of the package is that agentic inference needs a control plane for retained state, but only when that control plane exposes variables that affect future value. The validated runtime prototype turned this into an executable object registry and policy loop. It consumed the synthetic trace, reconstructed object lifetimes, and emitted placement, retention, compression, and eviction decisions. The important result was not the synthetic score values. It was that Option A, Option B, and Option C can be represented as different control-plane boundaries over the same trace-visible state.

For Option A, the control plane can remain coarse. The runtime boundary is the request, model, active context, and KV-cache page. This path is still correct for controls, batch/offline inference, cheap recomputation, single-turn use, and regimes where richer metadata has no causal value. In those regimes, exposing object registries, provenance pointers, branch IDs, verifier IDs, durable horizons, or trajectory edges would add queues and validation overhead without preserving additional retained value.

For Option B, the runtime needs a memory-object application binary interface (ABI). An ABI is the contract that lets independently implemented runtime, scheduler, cache, tool, and telemetry components agree on the identity and meaning of a retained object. In this package, an Option B object must carry enough metadata to decide whether reuse is useful and safe: object class, size, tier, lifetime, reuse count or predicted reuse probability, correctness sensitivity, provenance pointer, source version, invalidation state, tenant or cache-salt scope, validation

decision, compression boundary, and recoverability. These fields are not decorative. Runtime ablations showed that hiding provenance and reuse collapses RAG from Option B to Option A.

For Option C, the ABI must also describe graph structure. The required fields include trajectory node, branch ID, verifier ID, merge state, replay authorization scope, verifier evidence binding, durable workspace pointer, retention hold state, and dependency edges among branch, verifier, tool-output, replay, and durable artifacts. The package uses a directed acyclic graph (DAG) for those dependencies because an agent run can fork, verify, discard, merge, and replay state without forming a simple linear sequence. Runtime ablations showed that hiding branch, verifier, and durable fields collapses agentic workloads from Option C toward Option B or A.

The compiler and planner implication is a memory-planning pass over workflows, not only over kernels. The package's constrained planner consumes security decisions, runtime policy rows, compression safety scores, queueing thresholds, contention thresholds, and energy/contention collapse rows. Its planning value is:

```
NetPlanValue =
  SafeReuseValue
  - MovementCost
  - ValidationOverhead
  - QueueOverhead
  - ContentionPenalty
  - CompressionRisk
```

Security and compression safety are hard gates. Denied reuse forces recomputation or discard with zero positive reuse credit. Unsafe compression cannot be selected. Capacity, queueing, validation overhead, and contention are soft constraints that can downgrade Option C to Option B or A, offload cold state, preserve pointers, or mark a row infeasible. In the validated planner context, action rows included recompute/drop, offload cold state, keep-hot state, and compress-or-pointer-preserve decisions. The binding constraints were led by security gates, then contention tails, control or zero-reuse cases, capacity, validation overhead, queueing overhead, value positivity, and compression safety.

This implies compiler support beyond ordinary context-window packing. A memory-centric agent compiler or planner would need to allocate context budget, decide whether a tool output should be retained as an object or recomputed, place state across hot and warm tiers, preserve exact state when a summary would break replay, bind verifier artifacts to later branches, schedule branch-aware and verifier-aware retention, and emit durable replay plans when future recovery value exceeds durable-tail cost. It would also need explicit collapse paths. If the object metadata is unavailable, stale, unauthorised, too expensive to validate, or unjoinable with telemetry, the planner must fall back to the coarser architecture boundary.

ABI Control Plane

The ABI control plane is therefore a fail-closed sequence:

```
memory-object contract validation
  -> runtime/planner compatibility check
  -> security/provenance/retention/verifier gate
  -> placement, reuse, compression, migration, or retention action
  -> telemetry and evidence artifact emission
```

Rejected contracts stop before reuse or placement credit is granted. This is the control-plane interpretation of the production evidence chain. The same object identity, provenance, timebase, tenant, security, and dependency fields that let a runtime make decisions are also the fields future production telemetry must preserve to make those decisions auditable. The baseline control-plane figure remains the compact visual summary of this progression; the underlying progression table is `data/architecture_control_plane_progression.csv`:

The production-facing implication is that the ABI cannot be only an internal runtime API. It must be observable. The production telemetry contract, deployment blueprint, adapter conformance kit, intake bundle, trust policy, evidence gatechain, timebase checks, redaction integrity checks, causal-control gate, production replay surface, live collector preflight, and claim-expiry harness all exist because a memory-centric runtime claim is not testable unless the control plane emits joinable evidence. A future target deployment must join object identity, workload

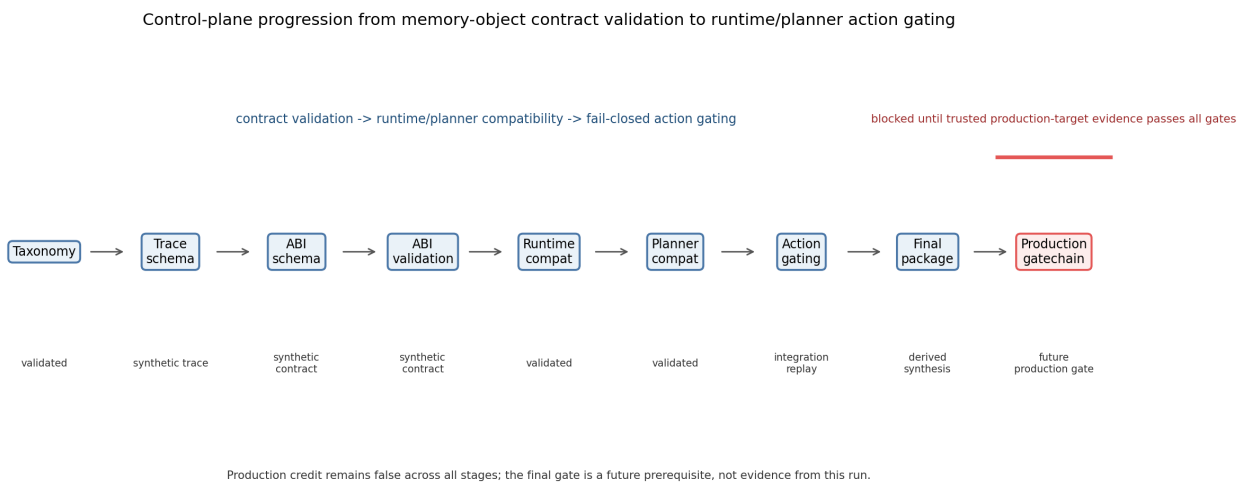

Figure 1: Control-plane progression from memory-object contract validation to runtime/planner action gating, with production-credit boundary shown as a blocked downstream path.

identity, topology, tenant context, security context, byte movement, power or cost counters, CXL or pooled-memory latency, queue depth, validation decisions, architecture option, and claim lifecycle state in the same replayable evidence path.

The result is a layered architecture rather than a single replacement for serving systems. Option A is the low-overhead serving substrate. Option B adds an object registry, safe object reuse, and object-local placement or compression. Option C adds trajectory/DAG state and graph-aware retention, replay, and merge semantics. The runtime must choose the coarsest layer that preserves the causal state variables for the workload. The compiler and control plane must then prove that the chosen layer remains cheaper, safer, and more useful than falling back.

Architecture Package and Reproduction Surface

The workspace package is an inspectable research package, not a closed-world production deployment package. Its substantive artifacts fall into four groups: narrative models under `memory-centric-agentic/`, executable scripts and tests under `scripts/` and `tests/`, generated data and figures under `data/`, and handoff/readiness artifacts that connect claims to reproduction commands. The final audit reports 89 validated milestones, 2 superseded milestones, and 1 in-progress sentinel. It also reports 107 figures present, all 107 represented in the ledger, 40 milestones with figures, and no missing or orphan figures.

The model layer contains the human-readable architecture record. It includes the workload taxonomy, lifetime model, cost model, simulator design, scheduling-abstraction note, architecture proposal, trace schema, queueing model, compression model, runtime prototype note, calibration map, security/provenance model, final synthesis, energy/economics/contention note, production telemetry contract, deployment blueprint, adapter and intake notes, trust-policy note, gatechain note, timebase and redaction notes, causal attribution note, production-target replay note, live collector preflight, claim expiry/revalidation note, and final architecture package. These files define the terms used in the report and explain the evidence labels attached to each result.

The executable mechanism layer contains the symbolic and synthetic machinery. Wolfram Language scripts generate lifetime, heterogeneous-cost, queueing, and compression boundary outputs. Python scripts generate synthetic workloads, compare memory policies, evaluate scheduling abstractions, generate and validate trace v2, simulate queueing overheads, evaluate compression strategies, replay the runtime prototype, build calibration maps, evaluate security/provenance, synthesize the research agenda, sweep energy/economics and CXL contention, and replay trace-v3 security enforcement. The tests verify that the generated outputs preserve the intended boundaries, such as no unsafe lossy compression for correctness-sensitive state, no unsupported queue-help labels, expected Option A/B/C runtime ablations, and zero positive credit for unsafe reuse.

The production-evidence layer contains the later gate work. It includes host-local DC-001/DC-002 proxy calibration, production-shaped DC-001/DC-002 telemetry ingestion, the deployment kit, future-trend falsification, adapter conformance, production intake, operator trust policy, gatechain replay, timebase integrity, redaction integrity, causal attribution, production-target replay, gate-evidence artifact validation, live collector preflight, and claim expiry. The common pattern is fail-closed: fixtures, proxy measurements, conformance rows, and dry-run rows may validate schema or replay behavior, but they do not grant production calibration or claim credit.

The baseline handoff artifacts remain useful for navigating the package. The handoff artifact index maps artifacts to type, producer, verifier, milestone, evidence class, dependencies, production-readiness impact, and limitations. The claim traceability table links final claims to data, narrative, validation, and figure artifacts. The reproduction manifest orders commands so a reader can regenerate the package without reconstructing the research sequence. The dependency and traceability figures preserve that map:

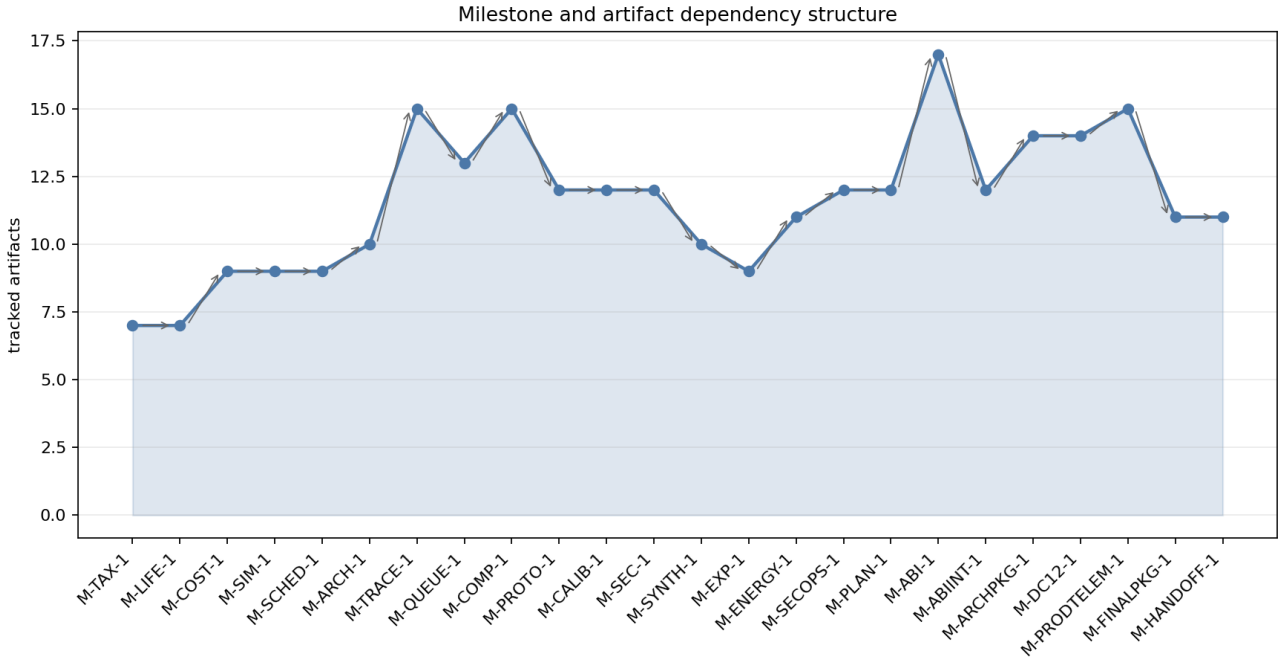

Figure 2: Milestone and artifact dependency structure.

The current reproduction surface is therefore strong for mechanism inspection. A reviewer can inspect the taxonomy, equations, synthetic traces, runtime replay, security replay, production-gate fixtures, generated CSVs, figures, and verifier scripts. The same reviewer should not treat the package as a sealed product artifact. The final audit identifies residual package debt: the handoff artifact index is internally valid but incomplete relative to the latest validated plan milestones and ledger artifact paths, and registered package archives contain only four files while pointing readers to final report files not present in the archive. Those are packaging and governance gaps, not new technical evidence for or against the architecture thesis.

The practical reproduction entry points remain:

```
python3 scripts/build_final_architecture_package.py
python3 scripts/plot_final_architecture_package.py
python3 scripts/build_architecture_control_plane_progression.py
python3 scripts/plot_architecture_control_plane_progression.py
python3 scripts/build_campaign_handoff.py
python3 scripts/plot_campaign_handoff.py
python3 tests/verify_memory_object_abi.py
python3 tests/verify_memory_object_abi_integration.py
python3 tests/verify_final_architecture_package.py
python3 tests/verify_campaign_handoff.py
```

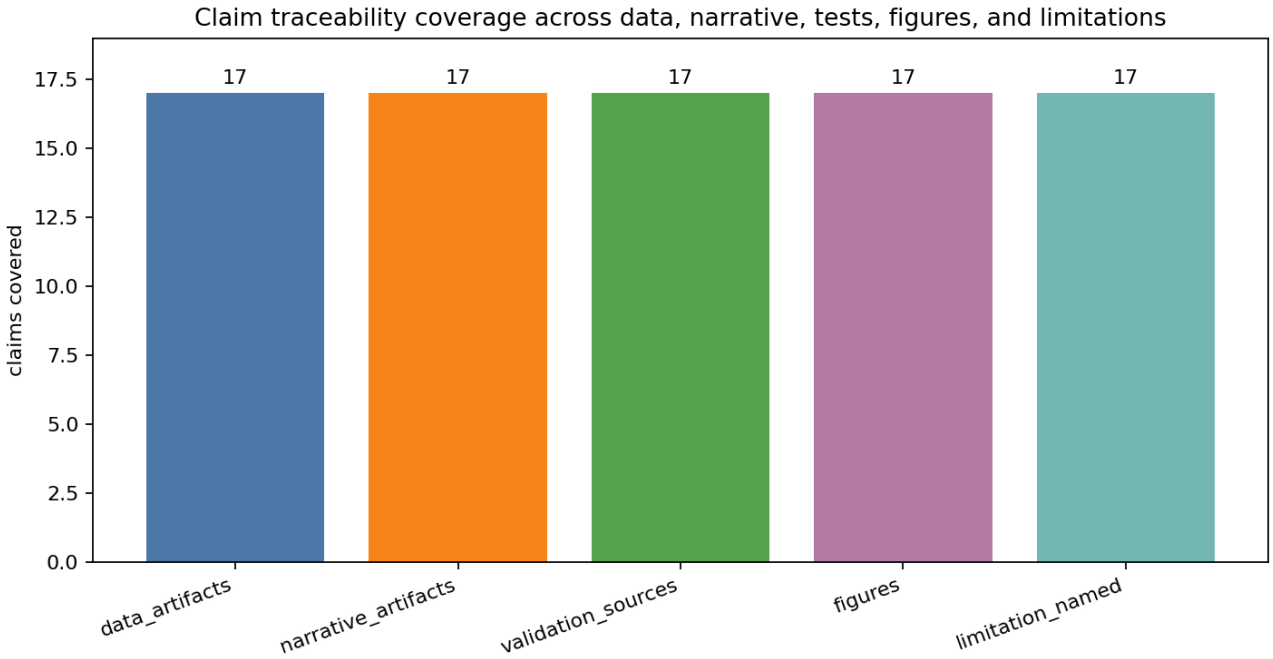

Figure 3: Claim traceability coverage across data, narrative, tests, and figures.

```
python3 tests/verify_architecture_control_plane_progression.py
python3 -m long_exposure.tools.promise_check .
python3 -m long_exposure.tools.org_check .
```

The final audit changes how this runbook should be interpreted. The mechanism and figure coverage are broadly validated, but `promise_check` is currently red because of a malformed ledger event and supersession metadata defect. A reader should therefore treat the command list as the intended closure surface from the baseline handoff, while the final residual-debt section records the governance repair needed before the package can be described as fully closed.

The package's value is that it makes the memory-centric architecture thesis falsifiable and reproducible at the research level. It provides equations, traces, policy comparisons, gate contracts, replay harnesses, and readiness matrices. It does not contain real joined production telemetry, and its registered archives are not yet complete handoff packages. That distinction is central to the final result: the architecture mechanism is inspectable, the production evidence contract is specified, and the remaining work is to repair governance/package debt and run the contract against real target deployments.

Falsification Criteria

The package defines falsification as an architecture-selection outcome, not as a single failed benchmark. A memory-centric design is rejected or downgraded when the retained value of state does not survive the costs of movement, residency, coordination, validation, security, compression, recovery, contention, and causal measurement. The baseline handoff already stated that the package fails if claims lack data, narrative, and validation traceability; if reproduction commands reference missing artifacts; if synthetic fixtures or host-local proxies are labeled production-ready; if security-denied reuse receives positive reuse, energy, or cost credit; if controls favor Option B or C without retained value; or if Option B or C remain preferred after queueing, contention, validation, or compression thresholds cross the validated reversal boundary.

For Option A, falsification works in the opposite direction. Option A remains the correct default when workloads have zero durable reuse, cheap recomputation, low branch fanout, low verifier or tool-output retention value, high object-registry overhead, untrusted evidence, unjoinable telemetry, or safe retained value that is erased by security and validation costs. If those conditions dominate, adding a memory-object registry or trajectory/DAG

fabric is unnecessary overhead.

For Option B, the decisive question is whether object-local reuse is safe, fresh, authorized, and net positive. Option B collapses back to Option A when retrieved context, prefix/cache objects, semantic-cache entries, or tool outputs do not show positive safe reuse value after lookup, validation, invalidation, provenance, tenant isolation, transfer, queueing, and recovery costs. The future-trend harness records a synthetic minimum reuse-probability threshold of 0.2 for Option B; below that threshold, object reuse is likely transient or workload-specific. Semantic-cache hit rate alone is not enough: a row must preserve safe valid value after wrong-reuse risk, invalidation, and recovery cost.

For Option C, the decisive question is whether graph-aware state is worth managing. Option C collapses to Option B or A when branch state, verifier state, durable workspace state, trajectory logs, or multi-agent merge state do not retain enough value to pay for DAG coordination, verifier synchronization, durable-tail latency, recovery, validation, security, and replay overhead. The future-trend harness records synthetic thresholds of minimum branch fanout 2 and durable lifetime 6 for Option C. If branch fanout remains below that level, if durable state is short-lived, or if verifier and replay artifacts cannot be bound to later decisions, the trajectory/DAG fabric does not carry its cost.

Energy and economics claims have separate falsifiers. The package does not currently report measured production energy savings. DC-001 is falsified for production purposes if measured per-tier energy or dollar savings per byte moved or retained are zero, below noise, or causally attributable to confounded controls rather than memory-centric placement. DC-002 is falsified if CXL or pooled-memory p95/p99 tails exceed the retained-value margin. The synthetic trend harness records a CXL p99 collapse threshold at 480x and a recompute-cost threshold of 0.5x; those are sensitivity thresholds, not production constants. The energy/contention harness and host-local proxy validate replay mechanics, not GPU/HBM/CXL/datacenter savings.

Compression and offload claims are also bounded. Compression is valid as a representation, capacity, movement, provenance, or local-storage tool only when it is safe for the object class and does not destroy correctness-sensitive replay. Current repaired compression results do not support a general claim that compression improves object-level queue thresholds. Any future row that grants queue-help credit to unsafe or unsupported compression would contradict the validated boundary.

The production evidence path adds claim-level falsifiers. Production endorsement is blocked unless real `production_target` telemetry passes intake custody, adapter conformance, normalization, trusted deployment-root and attestation gates, operator trust policy, timebase integrity, redaction and replay-identifiability checks, security/provenance/retention/verifier gates, noise-floor checks, threshold replay, uncertainty qualification, causal Option A control validity, planner/readiness boundaries, evidence artifact validation, live collection source-material checks, handoff traceability, and claim-expiry or revalidation policy. Malformed timing is `measurement_invalid`, not a negative threshold result. Under-redacted exports fail as `privacy_leakage`; over-redacted or malformed exports fail as `replay_nonidentifiable`. Robust statistical effects remain blocked when the Option A control is confounded or unidentified.

The current production-target replay reports zero real production-target manifests, zero claim-support candidates, and zero rows with `production_calibrated=true`, `production_ready=true`, or `claim_credit_allowed=true`. All 19 gatechain replay paths keep claim credit false. Live collection currently emits only dry-run fixture artifacts, and claim-expiry fixtures do not create current production credit. These are not failures of the mechanism stack. They are the criteria that prevent non-production evidence from being misreported as production endorsement.

Residual Debt and Future Work

The final audit status is mostly validated but not closed cleanly. The audit headline is: 89 validated milestones, 2 superseded milestones, 1 in-progress sentinel, findings at 1 critical, 14 moderate, and 2 minor, and `promise_check=red`. The wall cap was not hit. Figure coverage is complete in the audited ledger view: 107 figures are present, all 107 are represented in the ledger, 40 milestones have figures, and there are no missing or orphan figures.

The red `promise_check` is residual governance and package debt, not a new technical rejection of the memory-centric mechanism stack. The final audit records five residual-debt anchors:

Anchor	Debt	Reported effect
`_manager/validator-warnings`	`promise_ledger.jsonl` line 220 has a non-UUID `event_id` and a superseded event missing `supersedes`.	`promise_check` exits red.
`_run/report_cycles_*`	Older periodic reports preserve historical no-artifact, no-ledger, action-required, or integration-gap statements contradicted by later validated ledger events.	Public record needs supersession context.
`M-HANDOFF-1`	The handoff artifact index is internally valid but incomplete relative to the current 41 validated plan milestones and current ledger artifact paths.	Handoff navigation is stale relative to the final validated state.
`_run/final-package-artifacts`	Registered package archives contain only four files and point readers to final report files not present in the archive.	The archive is not a closed-world handoff package.
`_run/start`	The run-start sentinel remains `in-progress/high`.	This is not a plan-milestone completion gap, but it can imply active work remains open.

The audit-supplied future work is correspondingly bounded:

1. Repair ledger line 220 with a valid UUID, a correct `supersedes` pointer, and evidence-consistent narrative; rerun `promise_check` before closure.
2. Publish a consolidated errata or supersession index for stale periodic-report statements contradicted by later validated ledger events.
3. Regenerate the handoff artifact index against the latest plan and ledger, and extend its verifier to check milestone and artifact-path completeness.
4. Rebuild package archives as closed-world handoff artifacts with all referenced final report and verification files included.
5. Close or explicitly classify the run-start sentinel to avoid implying active research work remains open.

The technical future work is the production experiment already defined by the package rather than another synthetic layer. The next evidence-bearing step is real joined `production_target` telemetry under a trusted deployment root, replayed through the full gate path. The first production measurements should target the deferred constants and trend priorities already identified: joined reuse probability by object class, CXL or pooled-memory p99 under tenant concurrency, validation/security/provenance overhead per safe reuse, branch fanout and merge/discard rates, durable workspace lifetime and replay frequency, target accelerator energy per tier byte moved, and recompute cost for verifier and tool-output regeneration.

Conclusions

The final result supports a conditional memory-centric architecture thesis for agentic LLM inference. The work shows that agentic runs can create memory objects whose value is not captured by arithmetic throughput alone: retrieved context, prompt prefixes, semantic-cache entries, tool outputs, verifier state, branch state, trajectory logs, durable workspace state, and multi-agent merge state. When those objects have safe retained value, a runtime that can identify, place, reuse, validate, compress, retain, or replay them can be the right abstraction.

The thesis is not universal. Option A, conventional request/model/KV-centric serving, remains the validated baseline and control. It is the correct architecture for single-turn, batch/offline, cheap-recompute, zero-reuse, low-branching, high-overhead, untrusted, or unmeasured regimes. Option B is a mechanism-validated, contract-ready path for memory-object-aware reuse. Option C is a mechanism-validated, contract-ready path for trajectory/DAG-aware branch, verifier, durable, and replay state. Neither Option B nor Option C is production-endorsed by the

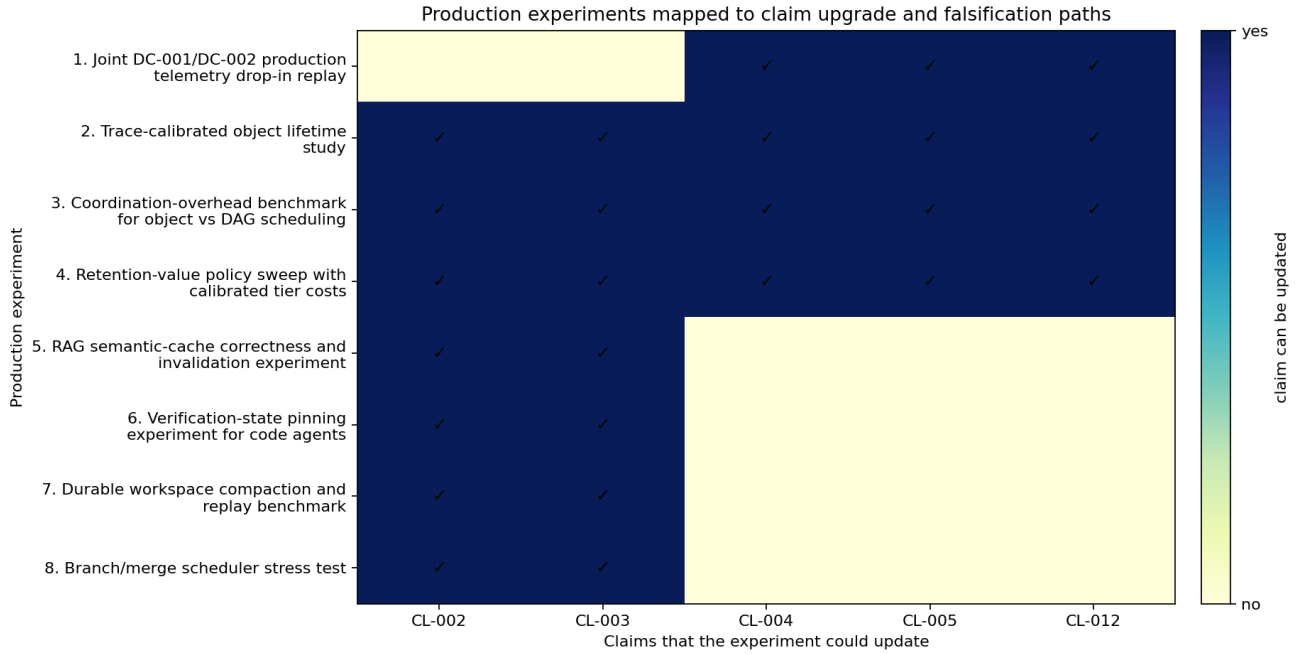

Figure 4: Production experiments mapped to claim upgrade and falsification paths.

current package.

The package's main contribution is the combination of mechanism and boundary. On the mechanism side, it supplies a taxonomy, lifetime and cost equations, synthetic policy comparisons, scheduling-abstraction reversals, trace replay, queueing thresholds, compression boundaries, runtime ablations, security-adjusted synthesis, and control-plane implications. On the boundary side, it supplies production telemetry contracts, deployment and adapter gates, intake custody, trust policy, gatechain replay, timebase and redaction checks, uncertainty and causal gates, production-target replay, evidence-artifact validation, live collection preflight, and claim-expiry rules. The same structure that makes Option B/C plausible also prevents premature production claims.

The final audit confirms broad validation of the research package while preserving a red closure status for governance and packaging defects. That distinction matters. The red `promise_check` means the ledger/package handoff needs repair before the run can be described as cleanly closed. It does not convert synthetic fixtures into production evidence, and it does not invalidate the mechanism-level architecture result.

The final architecture stance is therefore conservative and actionable: deploy Option A as the default and control; use Option B only when safe object reuse is observable, joinable, and net positive; use Option C only when branch, verifier, trajectory, durable, and replay state preserve value after all overheads and gates; and require real production-target telemetry plus lifecycle revalidation before claiming production energy, latency, cost, capacity, or readiness benefits.

References

[1] NVIDIA, "NVIDIA H100 Tensor Core GPU," NVIDIA Data Center, accessed 2026-05-11. Link: NVIDIA H100 page.

[2] NVIDIA, "NVIDIA H200 Tensor Core GPU," NVIDIA Data Center, accessed 2026-05-11. Link: NVIDIA H200 page.

[3] NVIDIA, "NVIDIA DGX B200: The Foundation for Your AI Factory," NVIDIA Data Center, accessed 2026-05-11. Link: NVIDIA DGX B200 page.

[4] NVIDIA, "Introduction to NVIDIA DGX H100/H200 Systems," NVIDIA DGX H100/H200 User Guide, accessed 2026-05-11. Link: DGX H100/H200 user guide.

[5] PCI-SIG, “PCI Express 6.0 Specification,” PCI-SIG, accessed 2026-05-11. Link: PCI Express 6.0 specification page.

[6] NVM Express, “Specifications,” NVM Express, accessed 2026-05-11. Link: NVM Express specifications.

[7] Compute Express Link Consortium, “CXL Specification,” Compute Express Link, accessed 2026-05-11. Link: CXL specification page.

[8] MLCommons, “MLPerf Inference: Datacenter Benchmark,” MLCommons, accessed 2026-05-11. Link: MLPerf Inference datacenter benchmark.

[9] Woosuk Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” arXiv, 2023. Link: arXiv:2309.06180.

[10] Intel, “Intel Xeon 6 Processors with MRDIMM - Solution Brief,” Intel, accessed 2026-05-11. Link: Intel Xeon 6 MRDIMM solution brief.

[11] AMD, “AMD EPYC 9005 Processor Architecture Overview,” AMD, accessed 2026-05-11. Link: AMD EPYC 9005 architecture overview.

[12] vLLM Project, “Automatic Prefix Caching,” vLLM Documentation, accessed 2026-05-11. Link: vLLM prefix caching documentation.

[13] Sajal Regmi and Chetan Phakami Pun, “GPT Semantic Cache: Reducing LLM Costs and Latency via Semantic Embedding Caching,” arXiv, 2024. Link: arXiv:2411.05276.

[14] CXL Consortium, “CXL Consortium Releases Compute Express Link 2.0 Specification,” Business Wire, 2020. Link: Business Wire CXL 2.0 release.